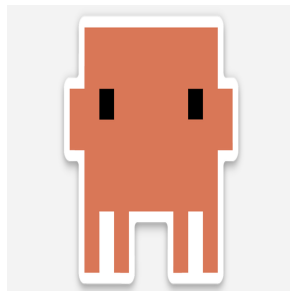




Claude Code Deep Dive

The Complete Guide to Understanding How It Works



Master the 9 essential elements that make Claude Code powerful

By Mark Kashef

AI & Automation Educator

Table of Contents

1. Introduction: Why This Guide Exists	3
2. The Big Picture: Claude Code Architecture	4
3. The Gather-Act-Verify Loop	6
4. Understanding the Tool System	8
5. File Operations: Read, Write, Edit	10
6. Search Tools: Glob and Grep	12
7. Bash: Command Execution Power	14
8. Context Management: The Bucket Analogy	16
9. Session Management	19
10. Customization: Claude.md, Skills & MCP	21
11. Expert Tips & Best Practices	23
12. Quick Reference Cheat Sheet	25

1. Introduction: Why This Guide Exists

This guide is the result of hours of deep investigation into how Claude Code actually works under the hood. The goal is simple: to help you understand Claude Code better than 99% of users out there, whether you're technical or non-technical.

When you understand the machinery behind the curtain, you gain the ability to:

- Work more efficiently by knowing what Claude Code can and cannot do
- Troubleshoot issues faster by understanding the underlying systems
- Optimize your workflow by leveraging the right tools at the right time
- Preserve your context window and avoid common pitfalls
- Customize Claude Code to fit your specific needs

The Secret: Interview Claude Code Itself

One of the best ways to understand Claude Code is to ask it directly. Claude Code comes with a built-in subagent called the **Claude Code Guide** that's specifically designed to answer questions about how the CLI, Agent SDK, and Claude API work.

@claude-code-guide Can you explain how context management works?

PRO TIP: Use **@claude-code-guide** whenever you have deep questions about how Claude Code works. It can create diagrams, explain systems, and help you understand the nuts and bolts.

2. The Big Picture: Claude Code Architecture

At its core, Claude Code consists of four main components working together:



Component Breakdown:

Terminal Interface: The command-line interface where you interact with Claude Code. This is the familiar box where you type prompts and see responses.

Session Manager: Handles the lifecycle of your conversations, including starting new sessions, maintaining state, and managing conversation history.

Tool Executor: The engine that runs all the tools Claude Code uses - reading files, searching code, running commands, and more.

Permission Layer: The safety system that asks for your approval before Claude Code performs potentially impactful actions like editing files or running commands.

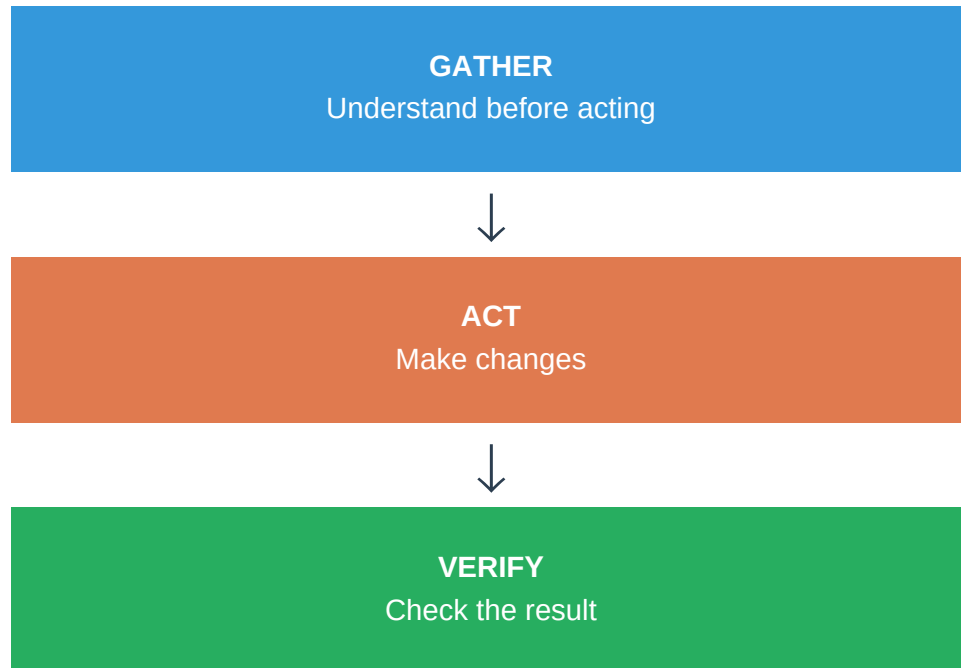
The Magic Ingredient: Intelligence + Open Source

Here's what most people don't realize: the core infrastructure of Claude Code (React, TypeScript, Node.js) is built on open-source technologies that have existed for years. Anthropic didn't invent these components - they orchestrated them brilliantly with Claude's intelligence.

NOTE: Claude Code is a combination of smart engineering standing on the shoulders of open-source giants. The real innovation is how everything comes together with actual AI intelligence.

3. The Gather-Act-Verify Loop

If you want one single diagram to understand what Claude Code does behind the scenes, it's the **Gather** → **Act** → **Verify** loop. This is the fundamental cycle that powers every interaction.



Stage 1: GATHER

In the gather stage, Claude Code understands before acting:

- Reads files and explores code structure
- Searches for relevant patterns and keywords
- Reads your Claude.md command center file
- Asks proactive questions if your prompt is vague

Stage 2: ACT

In the act stage, Claude Code makes changes:

- Edits existing files with surgical precision
- Creates new files and folders
- Runs Bash commands on your computer

-
- Installs packages and dependencies

Stage 3: VERIFY

In the verify stage, Claude Code checks its work:

- Runs automated checks and tests
- Reads the result of its changes
- Loops back if something went wrong
- Commits successful patterns to memory

PRO TIP: Keep these three words in mind: **Gather, Act, Verify**. This mental model helps you understand what Claude Code is doing at any moment and why.

4. Understanding the Tool System

Claude Code has different categories of tools - think of them as specialized workers, each with a specific job. Understanding these tools helps you work more effectively with Claude Code.

Category	Tools	Purpose
File Reading	Read	Observe and check code before acting
File Editing	Write, Edit	Create and modify files
Search	Glob, Grep	Find files and search content
Execution	Bash	Run commands on your computer
Web	WebSearch, WebFetch	Search and retrieve web content
Orchestration	Task, Skills	Manage workflows and subagents

The Tool Selection Process

When you send a request, Claude Code's brain (powered by the Claude model) decides which tools to use. Here's the decision process:

- **Need to read something?** → Use the Read tool
- **Need to find code?** → Use Grep (search content) or Glob (find files)
- **Need to change a file?** → Use the Edit tool
- **Need to run a command?** → Use the Bash tool
- **Need to search the web?** → Use WebSearch or WebFetch

Tool Results Create Context

Every time Claude Code executes a tool, the result creates new context - a new understanding of reality. If it's something important that needs to be remembered, Claude Code can commit that understanding to persistent memory (like your Claude.md file).

NOTE: *The tool system is designed to be memory-efficient. Claude Code tries to gather only the information it needs rather than loading everything at once. This preserves your precious context window.*

5. File Operations: Read, Write, Edit

These are Claude Code's core tools for working with files. Understanding their nuances helps you work more effectively.

The Read Tool

The Read tool lets Claude Code view file contents. It sees lines with numbers, which helps it reference specific locations when making changes.

WARNING: PDF Warning: *PDFs are notorious context hogs. They contain tons of noisy tokens for formatting that you don't see but Claude Code has to process. A 50-page PDF can instantly fill your context window.*

Expert Tip: Instead of having Claude Code read a huge PDF directly, create a Python script that uses a cheaper API (like Gemini Flash) to read and summarize it, then feed the summary to Claude Code.

The Write Tool

The Write tool creates new files from scratch. This is useful for:

- Creating new code files
- Generating markdown documentation
- Creating playbooks of successful workflows
- Committing compressed understanding to memory

PRO TIP: *After a successful session with lots of trial and error, ask Claude Code to create a playbook documenting what worked. This is reverse meta-prompting - capturing the successful path for next time.*

The Edit Tool

The Edit tool makes surgical changes to existing files. Instead of rewriting the entire file (which uses more tokens), it finds the exact text to replace and swaps it out.

1. Find old string



2. Replace with
new string



3. Verify change

This pinpoint editing approach saves tokens compared to regenerating entire files, which means you get more work done before hitting context limits.

6. Search Tools: Glob and Grep

Search is unbelievably important in Claude Code. These tools let you find files and content without loading everything into memory. This is key to preserving your context window.

Glob: Finding Files by Pattern

Glob searches for files by their names and patterns. When Claude Code needs to find all JavaScript files or all files in a specific folder, it uses Glob.

Pattern	What It Finds
<code>*.ts</code>	All TypeScript files
<code>src/**/*.js</code>	All JavaScript files in src and subdirectories
<code>**/*auth*</code>	Any file with 'auth' in the name

Grep: Searching Content

While Glob finds files by name, Grep searches inside files for specific text or patterns. Think of it like searching on a website or using Google - it finds where specific keywords appear.

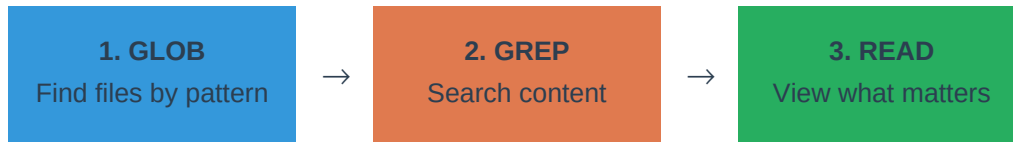
When you ask Claude Code to 'edit how we handle authentication,' it uses Grep to search for 'auth' across all files, finding every location where authentication code might live.

Powered by RipGrep

Behind the scenes, Grep is powered by RipGrep - an incredibly fast search tool. It's fast because it:

- **Runs searches in parallel** - searches multiple files simultaneously
- **Is memory efficient** - doesn't load entire files into memory
- **Respects .gitignore** - automatically skips irrelevant files

The Search → Read Workflow



PRO TIP: The key insight: Claude Code searches everything **before** reading to decide what's worth reading. This narrowing process preserves tokens by only loading the relevant parts.

7. Bash: Command Execution Power

Bash is what makes Claude Code so powerful for local tasks. It allows Claude Code to run commands on your computer - the same commands you could run in a terminal yourself.

What Bash Can Do:

- **Install packages** - npm install, pip install, etc.
- **Run tests** - verify code works correctly
- **Manage files** - organize, move, rename files and folders
- **Spin up servers** - run local development environments
- **Build applications** - compile and bundle code
- **Version control** - git commands for tracking changes

WARNING: Be careful with Bash! It can take full control of your computer - changing settings, permissions, even deleting things. Don't use YOLO mode until you understand what's happening.

Common Bash Patterns

Category	Commands	What They Do
Version Control	git status, git commit, git push	Track and save code changes
Package Management	npm install, pip install	Install dependencies
Building	npm run build, docker build	Compile and package code
Testing	npm test, pytest	Run automated tests

Quick Git Primer

If you're new to Git (version control), here's the crash course:

-
- **Repository:** A folder where Git tracks all your code changes
 - **Commit:** A checkpoint - a saved snapshot of your code at a point in time
 - **Branch:** A parallel version of your code for testing features safely
 - **Main/Master:** The primary branch - your 'production' code
 - **Push:** Upload your commits to a remote server (like GitHub)

PRO TIP: To set up GitHub in Claude Code, type **`/install GitHub`** and follow the wizard. It takes about 15-20 minutes but gives you version control for all your projects.

8. Context Management: The Bucket Analogy

This is arguably the most important section for understanding your limitations with Claude Code. Context management determines how much you can do in a single session.

The Bucket Mental Model

Think of Claude Code's working memory as a **bucket**. This bucket has a fixed size (around 200,000 tokens for Opus 4.5), and everything you do fills it up:

- Your Claude.md file loads at the start (can be 10-30% right away)
- Each message you send adds to the bucket
- Each file Claude Code reads adds to the bucket
- Tool results (especially JSON from MCP servers) add to the bucket
- PDFs are especially heavy due to hidden formatting tokens

Context Window Usage	
System Prompt	~5%
Claude.md	10-30%
Conversation History	Variable
File Contents Read	Variable
Tool Results	Variable

The 40-50% Threshold

Here's a critical insight: once you pass the **40-50% threshold** of your context window, Claude Code starts to degrade. You'll notice:

- Lower quality answers and code
- More 'lazy' responses instead of proactive help
- Forgetting earlier parts of the conversation
- More frequent mistakes

NOTE: Think of context like a physical budget. If you have \$200,000 and each request costs \$10,000, be thoughtful about what you spend. The beginning of a session is when Claude Code is most capable.

Check Your Context

Type **/context** in Claude Code to see your current context window usage. This shows you exactly how full your bucket is.

```
/context
```

Understanding Compaction

When your bucket gets full (100%), Claude Code automatically performs **compaction** - summarizing the conversation to free up space. This has implications:

- Details get lost in summarization
- Context from earlier in the conversation may disappear
- Multiple compactions compound the information loss
- You don't control what gets kept or dropped

Pre-Compaction Strategy

Before compaction happens, take control of what gets preserved:

Step 1: Ask Claude Code to write a summary of the most important points from your session

Step 2: Review and adjust the summary if needed

Step 3: Run **/compact** - it will weight your recent summary more heavily

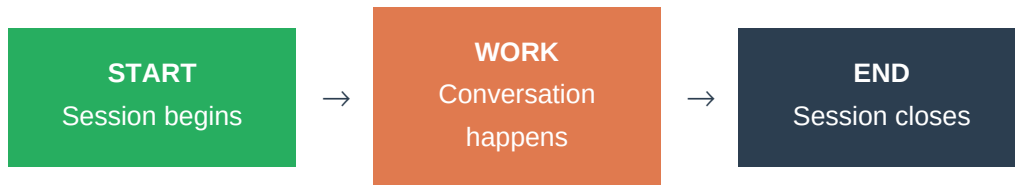
Step 4: Update your Claude.md with any persistent learnings

PRO TIP: If Claude keeps forgetting things across sessions, commit the important understanding to your **Claude.md** file. This persists across all sessions.

9. Session Management

Each terminal you open creates a new session. Understanding how sessions work helps you organize your work more effectively.

Session Lifecycle



Stateless Sessions

Sessions are **stateless** - each new session starts with a blank slate (except for your Claude.md and system prompt). When you close a session, you lose:

- The context window contents
- File snapshots from that session
- In-memory state (what files were read)

What persists:

- Files you created or modified
- Your Claude.md file
- Git commits
- Installed packages
- Conversation history (stored in ~/.claude folder)

File Snapshots

Before Claude Code edits a file, it takes a **snapshot** of the current state. This allows it to restore the original if something goes wrong. The process:

1. Look at current file state
2. Save snapshot in memory

-
3. Make the edit
 4. If something goes wrong → restore from snapshot
 5. If successful → update to new file state

PRO TIP: You can recover past conversations! All conversations are stored in the `~/.claude` folder. Ask Claude Code to search through them for any gold nuggets you want to persist.

Multiple Terminal Strategy

A powerful technique is to use **multiple terminals** for different tasks. This lets you:

- Keep separate context windows for separate concerns
- Avoid polluting one task's context with another
- Run long-running tasks in the background
- Use Plan Mode in one terminal while executing in another

10. Customization: Claude.md, Skills & MCP

Claude Code was designed to be hackable. Here are the five core things you can customize:

What	What It Does	When to Use
Claude.md	Command center that loads every session	Persistent instructions, project context
Skills	Mini workflows invoked on demand	Repeatable processes
MCP Servers	Plugins that add new capabilities	Integrations with external services
Hooks	Commands that run on events	Automation triggers
Settings	Configuration for behavior	Permission levels, defaults

Claude.md: Your Command Center

The Claude.md file is the brain that persists across sessions. It loads at the very beginning of every session, making Claude Code suggestible to your instructions when it's most helpful.

- Keep it focused - a bloated Claude.md wastes context
- Include project-specific patterns and conventions
- Document learnings from successful sessions
- Add instructions Claude Code frequently forgets

Skills: Just-in-Time Workflows

Skills are mini workflows that Claude Code invokes when needed. Unlike MCP servers that load at session start (using context), skills are **just-in-time** - only loaded when relevant.

Think of skills like Zapier or n8n workflows: they define a path from input A to output B, but with some AI flexibility built in.

MCP Servers: Use Sparingly

MCP (Model Context Protocol) servers add external capabilities like database access, API integrations, and more. However, they come with tradeoffs:

- Many are built once and never maintained
- Some have security issues
- They can bloat your context window significantly
- You might only need 1 tool out of 100 that load

NOTE: Start with 1-3 MCP servers maximum. The Supabase MCP is useful for database work, and Vercel for deployments. Add more only when you have a specific need.

11. Expert Tips & Best Practices

These tips come from extensive use of Claude Code and will help you avoid common pitfalls.

1. Use Plan Mode Strategically

Plan Mode lets Claude Code explore your codebase and create a plan without executing. The killer feature: you can **clear your context window** after planning, then start a fresh session to execute the plan with maximum context available.

2. Offload Heavy Reading

Instead of having Claude Code read large files directly, create scripts that use cheaper APIs (like Gemini Flash with 1M context) to read and summarize, then feed the summary to Claude Code.

3. Spin Up Subagents for Isolation

Subagents get their own fresh 200K context window. Use them for tasks that would pollute your main session, like reading large files or doing exploratory research.

4. Commit Understanding to Memory

After solving a tricky problem, ask Claude Code to commit its understanding as a compressed TLDR to your Claude.md file. This prevents having to re-discover solutions.

5. Use GitHub Issues as a Persistent To-Do List

If you don't want to bloat your Claude.md with task lists, use GitHub Issues. They live in the cloud and can be referenced at the start of any session.

6. Graduate Permission Levels Slowly

Start with 'Ask Before Edits' mode to learn what Claude Code does. This helps you understand the patterns before moving to YOLO/bypass mode. The permission prompts teach you.

7. Create Playbooks for Repeatable Workflows

After a successful session with trial and error, have Claude Code write a playbook documenting: what decisions were made, what worked, and how to go directly to the outcome next time.

NOTE: *The human-in-the-loop design is intentional. Claude Code was built to be your coding companion, not a fully autonomous agent. Work with it, not around it.*

12. Quick Reference Cheat Sheet

Essential Commands

Command	What It Does
/context	Check your context window usage
/compact	Manually trigger compaction
/install GitHub	Set up GitHub integration
@claude-code-guide	Ask the Claude Code expert subagent

The Core Loop

GATHER → ACT → VERIFY → (repeat)

Context Window Rules of Thumb

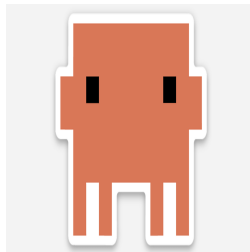
- Stay under 40-50% for best performance
- PDFs are context killers - offload to cheaper APIs
- Summarize before compacting to preserve what matters
- Use subagents for isolated heavy tasks
- Multiple terminals = multiple context windows

Tool Quick Reference

Need to...	Use
Find files by name/pattern	Glob
Search content in files	Grep
View file contents	Read
Change part of a file	Edit

Create new file	Write
Run terminal command	Bash

"Claude Code is a combination of smart engineering standing on the shoulders of open-source giants. The real magic is understanding how it all comes together."



Now go build something amazing!